

Lesson Plan: Building Agents with the DOE Framework

What You'll Learn

By the end of this lesson you'll be able to build a working AI agent from scratch using the DOE Framework — one that's reliable, self-healing, and easy to extend.

Lesson 0: The Basics (For Non-Technical People)

If you've never built software before, this section covers every concept you need. Read it once, refer back as needed. Once these click, the rest of the lesson plan is easy.

0.1 What Is an "Agent"?

An agent is software that can do work on your behalf, making decisions along the way.

- A **script** does one fixed thing every time you run it (like a recipe).
- An **agent** reads the situation, decides what to do, runs the right scripts, and handles problems — like a capable employee following an SOP.

In this framework, the "brain" is an LLM (like GitHub Copilot), and the "hands" are Python scripts. The agent reads your instructions (directives), thinks about what to do, and then runs the right scripts in the right order.

0.2 What Is an LLM?

LLM stands for **Large Language Model**. It's the AI that powers tools like ChatGPT, Claude, and GitHub Copilot.

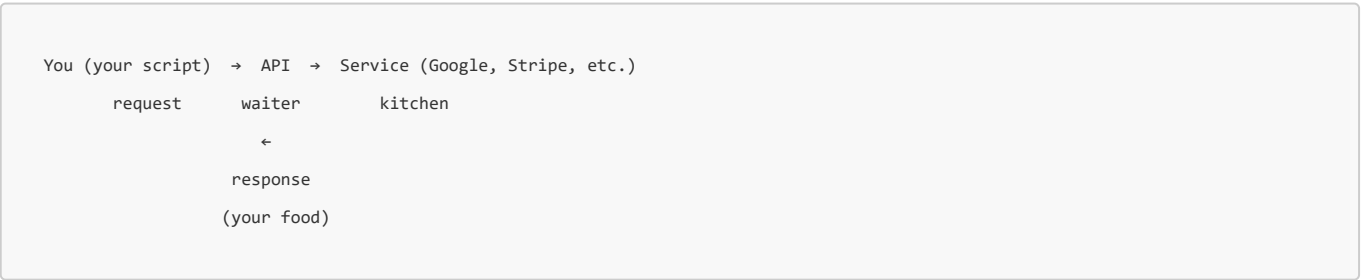
- It reads text, understands what you're asking, and generates useful responses.
- It's great at reasoning, writing, summarizing, and making decisions.
- It's **not** great at doing things repeatedly and consistently — that's why we pair it with scripts.

When this framework says "the orchestrator," it means the LLM — the AI brain making the decisions.

0.3 What Is an API?

API stands for **Application Programming Interface**. It's how two pieces of software talk to each other.

Think of a restaurant: you (the customer) don't walk into the kitchen. You tell the waiter what you want, the waiter tells the kitchen, and the kitchen sends back your food. The waiter is the API.



Examples of APIs you'll use:

- **Google Sheets API** — read/write spreadsheet data from a script
- **SerpAPI** — get Google search results as structured data
- **Apify API** — run web scrapers in the cloud
- **Instantly API** — manage email campaigns
- **OpenAI / Anthropic API** — send a prompt to an LLM and get a response back

Every external service your agent talks to, it talks to through an API.

0.4 What Is an API Key?

An API key is a **password that identifies your account** to a service.

When your script calls the Google Sheets API, Google needs to know who's asking. The API key proves it's you (and lets the service bill you or enforce your usage limits).

```
Without API key: "Hey Google, give me spreadsheet data" → ❌ Who are you?
With API key:   "Hey Google, here's my key: sk-abc123" → ✅ Here's your data
```

Key rules:

- **Never share API keys publicly** (not in GitHub repos, not in screenshots)
- **Store them in a `.env` file** — a special file that holds secrets
- **Each service has its own key** — you'll have one for SerpAPI, one for OpenAI, etc.

Getting an API key usually means: sign up for the service → go to settings/developer section → create a key → copy it into your `.env` file.

0.5 What Is a `.env` File?

A `.env` file is a simple text file that stores your secrets (API keys, passwords, tokens) in one place.

It looks like this:

```
SERPAPI_API_KEY=abc123yourkey
OPENAI_API_KEY=sk-xyz789anotherkey
GOOGLE_SHEETS_CREDENTIALS_FILE=credentials.json
```

Your Python scripts read from this file automatically, so you never hard-code secrets into your code.

Important: The `.env` file should **never** be uploaded to GitHub or shared. It stays on your machine only.

0.6 What Is JSON?

JSON (JavaScript Object Notation) is the universal format for structured data. Almost every API sends and receives JSON.

It looks like this:

```
{
  "company": "Acme Inc",
  "industry": "SaaS",
  "employees": 50,
  "locations": ["New York", "London"],
  "is_active": true
}
```

Think of it as a really organized way to write a list of facts. It uses:

- `{ }` for objects (a group of labeled values)
- `[ ]` for arrays (a list of items)
- `"key": "value"` for each piece of data

When a script saves results to `.tmp/results.json`, this is the format it uses. When another script reads that file, it knows exactly how to pull out the data it needs.

0.7 What Is Python?

Python is a programming language. In this framework, all execution scripts are written in Python.

You don't need to be a Python expert. You need to know:

1. **How to run a Python script:**

```
python execution/my_script.py --input "hello" --output .tmp/result.json
```

This tells Python to run the file `my_script.py` and pass it some inputs.

2. **What `--arguments` are:** Those `--input` and `--output` parts are called arguments. They tell the script what data to use and where to save results.

3. **What `pip install` does:** Python has a package manager called `pip`. When a script needs an extra library (like a tool to call Google's API), you install it with:

```
pip install google-api-python-client
```

The `requirements.txt` file lists all the packages an agent needs. Install them all at once:

```
pip install -r requirements.txt
```

0.8 What Is the Terminal (Command Line)?

The **terminal** is a text-based interface where you type commands to run things.

Instead of clicking buttons in a graphical interface, you type:

```
python execution/scrape_leads.py --industry "Plumbers" --location "New York"
```

In VS Code, the terminal is built in — it's the panel at the bottom of the screen. You'll use it to:

- Run Python scripts
- Install packages
- Check if things are working

Don't be intimidated by it. The terminal is just a way to give your computer precise instructions. Every command in this lesson plan can be copy-pasted directly into a terminal.

0.9 What Is OAuth?

Some APIs (especially Google's) use **OAuth** instead of a simple API key. OAuth is a more secure way to grant access.

The flow looks like this:

1. You set up a project in Google Cloud Console
2. You download a `credentials.json` file
3. The first time you run a Google script, a browser window opens asking you to log in and grant permission
4. Once you approve, it saves a `token.json` file so you don't have to log in again

You'll see OAuth referenced when working with Google Sheets and Google Docs scripts.

In practice: You set it up once, and then it just works. If it stops working, delete `token.json` and redo step 3.

0.10 What Is Web Scraping?

Web scraping is automatically extracting data from websites.

Instead of manually visiting 500 business listings and copying names and phone numbers into a spreadsheet, a scraper does it in minutes.

In this framework, scraping is handled by external services like **Apify** — you send them a search query, they run the scraper in the cloud, and send back structured data (JSON) with all the results.

You don't need to write scraping code yourself. The scripts in `execution/` handle the API calls to these services.

0.11 What Is a Webhook?

A webhook is the reverse of an API call. Instead of your script asking a service for data, **the service sends data to your script** when something happens.

```
Normal API call: Your script → "Hey, any new leads?" → Service
Webhook:         Service → "New lead just came in!" → Your script
```

Analogy: A regular API call is like checking your mailbox. A webhook is like the mailman ringing your doorbell.

Use webhooks when you want your agent to react to events in real-time (e.g., a new form submission, a new email reply).

0.12 What Is an MCP (Model Context Protocol)?

MCP is a standard that lets AI agents connect to external tools and data sources in a plug-and-play way.

Think of it like USB for AI:

- Before USB, every device had a different connector. Printers, scanners, cameras — all different plugs.
- USB gave us one standard plug that works for everything.
- **MCP does the same thing for AI tools.** Instead of every AI needing custom code to use Google Sheets, Slack, or a database, MCP provides one standard interface.

How it works in practice:

1. Someone builds an "MCP server" for a tool (e.g., Google Sheets, GitHub, a database)
2. Your AI agent connects to it using the MCP standard
3. The agent can now read/write data, call functions, and use that tool — without any custom integration code

Why it matters for this framework:

- MCP servers can serve as additional execution tools alongside your Python scripts
- Instead of writing a new script for every service, you can plug in an MCP server
- The AI orchestrator sees MCP tools the same way it sees scripts — as things it can call to get work done

You don't need MCP to get started. It's an advanced capability that becomes useful as you connect to more services.

0.13 What Is Git and GitHub?

Git is a tool that tracks changes to your files over time — like "Track Changes" in Word, but for code.

GitHub is a website that stores your Git projects online so you can share them, back them up, and collaborate.

Key concepts:

- **Repository (repo):** A project folder tracked by Git
- **Commit:** A saved snapshot of your changes (like a save point in a game)
- **Push:** Upload your commits to GitHub
- **Pull:** Download the latest changes from GitHub
- **.gitignore:** A file that tells Git what NOT to track (like `.env` and `token.json` — you never want secrets on GitHub)

For this framework, Git ensures you don't lose your work and can undo mistakes. GitHub stores the agent workspace so you can access it from anywhere.

0.14 What Is Markdown?

Markdown is a simple way to format text using plain characters.

All the directives in this framework are written in Markdown (`.md` files). Here's the syntax:

```
# Big Heading
## Smaller Heading
### Even Smaller

**bold text**
*italic text*

- bullet point
- another point

1. numbered item
2. another item

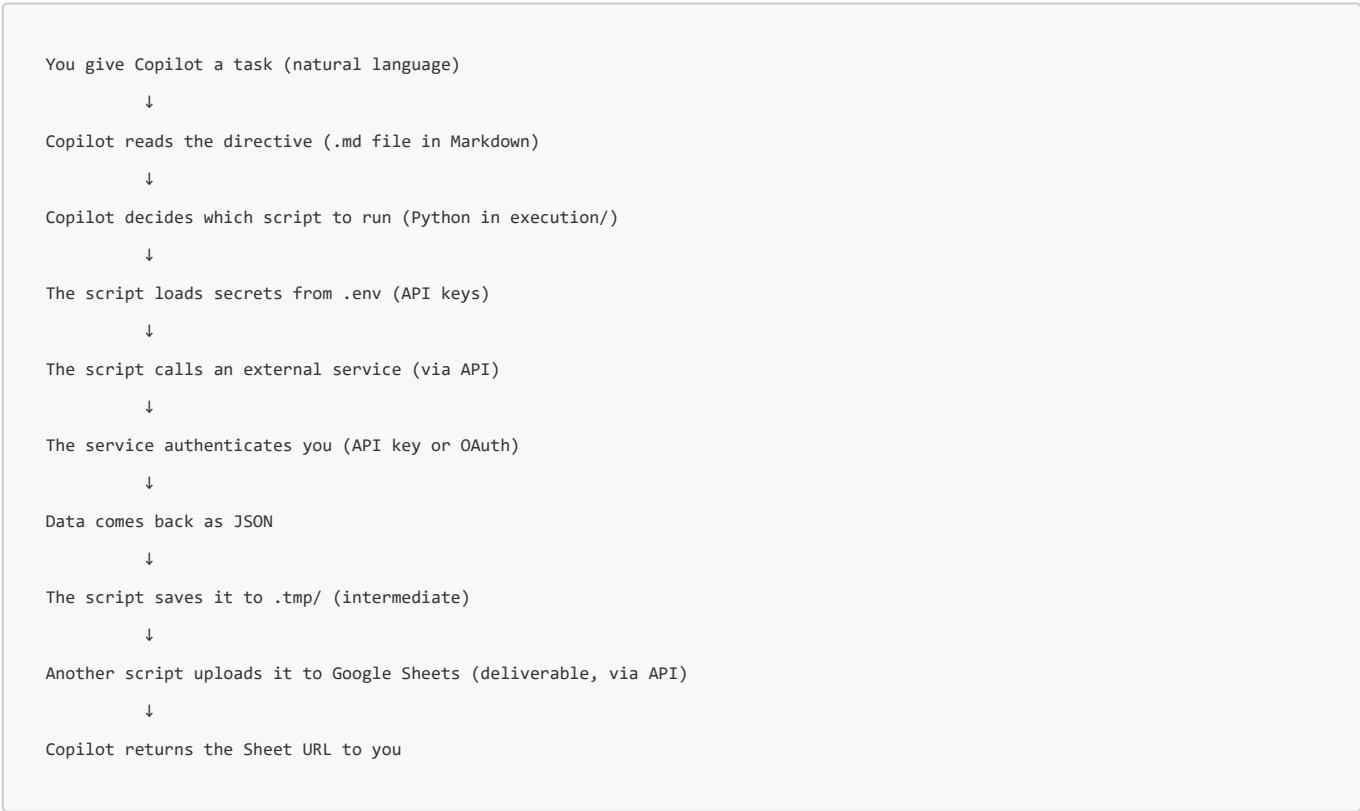
| Column 1 | Column 2 |
|-----|-----|
| data    | data    |

`inline code`
```

You're already reading Markdown right now — this lesson plan is a `.md` file. VS Code renders it nicely, but it's really just plain text with some formatting characters.

0.15 Putting It All Together

Here's how all these concepts connect when an agent runs:



Every concept from this section appears in that flow. Once you see how they connect, the rest of the framework is just details.

The Core Problem This Framework Solves

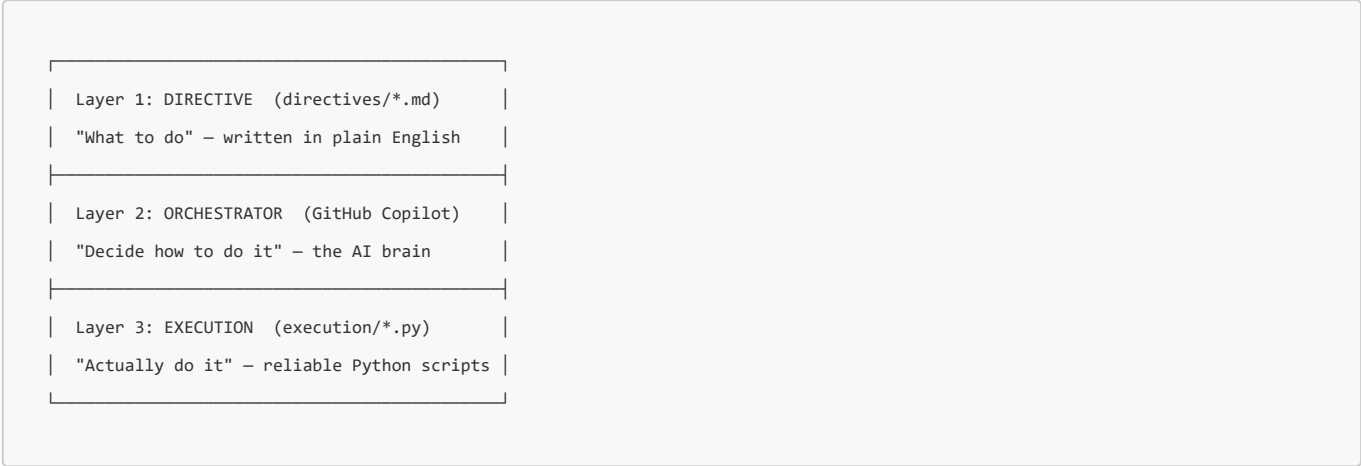
AI models are probabilistic. Business logic needs to be consistent.

If an AI does 5 steps and each step is 90% accurate, the end-to-end success rate is only **59%**. That's not usable for real work.

The DOE Framework fixes this by keeping AI in charge of *decisions* and handing all *execution* off to deterministic Python scripts.

Lesson 1: The 3-Layer Architecture

Every agent in this framework has exactly 3 layers. Learn these and everything else clicks.



Layer 1 — Directive

A Markdown file that acts like an SOP (Standard Operating Procedure). It answers:

- What is the goal?
- What inputs are needed?
- Which scripts exist to help?
- What is the step-by-step workflow?
- What are common edge cases?

Think of it as a job description for the agent.

Layer 2 — Orchestrator

This is GitHub Copilot (or another LLM). Its only job is intelligent routing:

- Read the directive
- Decide the order to run things
- Call scripts with the right inputs
- Handle errors
- Update the directive when it learns something new

The orchestrator never does grunt work directly. It delegates to scripts.

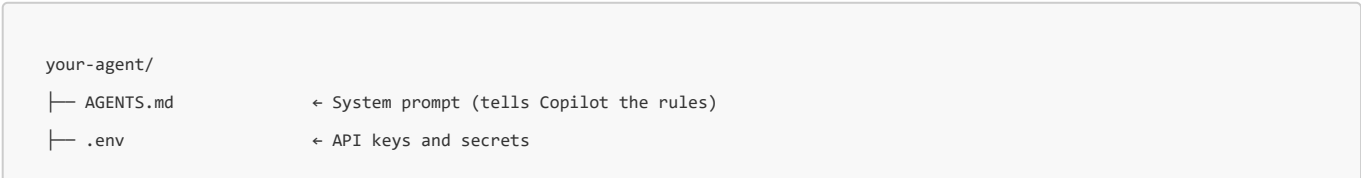
Layer 3 — Execution

Python scripts that do exactly one thing reliably. They:

- Call APIs
- Read/write files
- Process data
- Upload to Google Sheets / Docs

Scripts are deterministic — same inputs always produce the same outputs.

Lesson 2: The File Structure



```
├─ directives/
|   └─ my_task.md           ← Your SOP files
├─ execution/
|   └─ my_script.py        ← Your Python scripts
└─ .tmp/                   ← Temporary files (never commit)
```

Key rule: Local files are for processing only. Final deliverables live in the cloud (Google Sheets, Google Docs, etc.) where the user can access them. Everything in `.tmp/` can be deleted and regenerated.

Lesson 3: Anatomy of a Directive

Open any file in `directives/` and you'll see the same structure every time.

```
# Task Name

## Goal
One paragraph: what does this agent accomplish?

## Inputs


| Input        | Required | Description           |
|--------------|----------|-----------------------|
| Company Name | Yes      | Name of the business  |
| Industry     | No       | e.g. "SaaS", "Retail" |



## Tools/Scripts
- `execution/script_name.py` - what it does, what API key it needs

## Workflow
Step-by-step numbered instructions the orchestrator follows.

## Edge Cases
What to do when things go wrong.
```

That's it. No code, no configuration — just clear instructions.

Lesson 4: Anatomy of an Execution Script

Scripts follow a simple pattern:

```
# 1. Load environment variables
from dotenv import load_dotenv
load_dotenv()

# 2. Accept inputs via command-line arguments
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--input", required=True)
parser.add_argument("--output", required=True)
args = parser.parse_args()
```

```
# 3. Do one thing reliably
results = call_some_api(args.input)

# 4. Write output to a file (usually .tmp/)
with open(args.output, "w") as f:
    json.dump(results, f)

print(f"Done. Output saved to {args.output}")
```

Design rule: One script, one job. Keep scripts small and focused so they're easy to test and fix.

Lesson 5: How Copilot Thinks (The Orchestration Loop)

When you give Copilot a task, here's what it does internally:

1. READ the relevant directive in directives/
2. IDENTIFY what inputs are needed – ask the user if missing
3. RUN scripts in the order the directive specifies
4. CHECK the output of each script
5. DECIDE what to do next (or stop if done)
6. HANDLE errors (fix → retest → update directive)
7. DELIVER the final output to the user

Copilot never guesses how to do things — it reads the directive. This is why keeping directives accurate and up-to-date is so important.

Lesson 6: Self-Annealing (How the System Gets Smarter)

When something breaks, you don't just fix it and move on. You make the system so it never breaks the same way again.

```
Error occurs
  ↓
Read the error message
  ↓
Fix the script
  ↓
Test the fix
  ↓
Update the directive with what you learned
  ↓
System is now stronger
```

Example: A script hits an API rate limit.

- Fix: Add retry logic with exponential backoff
- Directive update: Add a note — *"This API allows 10 requests/minute. The script handles rate limits automatically."*

Now anyone (or any AI) reading the directive knows about this constraint upfront.

Lesson 7: Build Your First Agent (Step-by-Step)

Let's build a simple "Hello World" agent that scrapes some data and saves it to a Google Sheet.

Step 1 — Write the Directive

Create `directives/my_first_agent.md`:

```
# My First Agent

## Goal
Scrape the top 10 results for a search query and save them to a Google Sheet.

## Inputs
| Input      | Required | Description          |
|-----|-----|-----|
| Search Query | Yes      | The topic to search for |
| Sheet ID    | Yes      | Google Sheet to write output |

## Tools/Scripts
- `execution/search_web.py` - searches the web, outputs JSON
  - Requires: SERPAPI_API_KEY
- `execution/update_sheet.py` - writes data to Google Sheets
  - Requires: credentials.json

## Workflow
1. Run `search_web.py` with the query. Save to `.tmp/results.json`.
2. Read `.tmp/results.json` and verify results look correct.
3. Run `update_sheet.py` to write results to the Google Sheet.
4. Return the Sheet URL to the user.

## Edge Cases
- If fewer than 5 results are returned, ask the user to broaden the query.
- If the Sheet ID is invalid, provide a clear error message.
```

Step 2 — Write the Execution Script

Create `execution/search_web.py` that calls an API and saves output to a file.

Step 3 — Add API Keys to .env

```
SERPAPI_API_KEY=your_key_here
```

Step 4 — Test It

Ask Copilot: "Search for 'AI startups 2025' and save the results to my Google Sheet."

Copilot reads your directive → runs your scripts → delivers the output.

Lesson 8: Using the Pre-Built Agent Generator

You don't always need to start from scratch. This framework has a built-in generator:

```
python execution/create_agent_workspace.py --name "My Lead Gen Agent" --type lead_generation
```

Available agent types:

Type	What it does
lead_generation	Scrapes leads from Google Maps, SERP, enriches with emails
email_automation	Automates cold email via Instantly.ai
freelance_proposals	Scrapes Upwork jobs and generates proposals
video_editing	Automates jump cuts and video transitions
crm_integration	Manages data between sheets, webhooks, cloud services
research	Creates literature review papers from academic sources
full_stack	Everything above combined
custom	Minimal base for building anything from scratch

This generates a complete workspace in `outputs/` with the right scripts, directives, setup files, and README — ready to use.

Lesson 9: Two Ways to Deploy

Mode 1 — Copilot as Orchestrator (recommended for development)

- You work in VS Code with GitHub Copilot
- Copilot IS the AI — no LLM API keys needed for reasoning
- Scripts handle the deterministic work
- Great for: building, testing, iterating quickly

Mode 2 — Standalone / Cloud Deployment

- Agent runs on its own (cron job, webhook, cloud function)
- Scripts call LLMs directly using `OPENAI_API_KEY` or `ANTHROPIC_API_KEY`
- Great for: automation pipelines that run without a human present

The same directives and scripts work in both modes — you just change where the intelligence comes from.

Lesson 10: The Golden Rules

1. **Directives are living documents.** Update them every time you learn something new about an API, edge case, or better approach.
2. **Scripts do one thing.** Resist the urge to build a mega-script. Small, focused scripts are easier to fix and reuse.
3. **Copilot routes, scripts execute.** Never have Copilot manually process data it could hand off to a script.
4. **`.tmp/` is disposable.** Never store anything important there. Real outputs go to Google Sheets, Docs, or another cloud service.
5. **Self-anneal every time.** Fix → test → update directive. Don't just patch and leave.

Quick Reference

I want to...	I should...
-----	-----
Define what my agent does	Write a directive in directives/
Make the agent do something reliably	Write a script in execution/
Store an API key	Add it to .env
Save intermediate data	Write to .tmp/
Deliver output to the user	Write to Google Sheets / Docs

Create a new standalone agent	Run create_agent_workspace.py
Fix a recurring bug	Fix script + update directive

Lesson 11: How Humans Should Think When Working With Agents

Agents are only as good as the humans directing them. These mental models will make you dramatically better at designing, tasking, and debugging agents.

11.1 Deciding What to Automate — Jobs to Be Done

Before building anything, ask: **"What job is this agent hired to do?"**

People don't want a lead scraper. They want a pipeline of qualified prospects so they can close more deals. That's the job. The scraper is just the mechanism.

When you define the job clearly, the entire directive writes itself:

- **Goal** = the job to be done
- **Inputs** = what the job requires
- **Output** = the artifact that proves the job is complete

If you can't articulate the job in one sentence, the agent scope is too fuzzy. Keep narrowing until it's crisp.

11.2 Designing Agents — First Principles

Don't start by asking "what tools do I have?" Start by asking **"what does this task actually require, at its core?"**

1. What is the simplest possible input?
 2. What is the simplest possible output?
 3. What are the absolute minimum steps between them?

Then build exactly that. You can always add complexity later, but bloated agents are hard to debug and maintain.

Example: A proposal generator doesn't need to scrape 10 competitor sites, analyze sentiment, and build a market map. It needs: job description in → compelling proposal out. Start there.

11.3 Breaking Complex Tasks Into Smaller Steps

Agents fail when tasks are too big and vague. The fix is decomposition.

Rule of thumb: If a step in your directive can't be run as a single script call, it's too big. Break it down.

BAD: "Research the market and create a business plan"	
GOOD:	
Step 1: Search for market size data	→ serp_market_research.py
Step 2: Search for competitor info	→ serp_market_research.py
Step 3: Generate SWOT from the findings	→ generate_business_plan.py
Step 4: Generate financial projections	→ generate_business_plan.py
Step 5: Compile into a Google Doc	→ create_google_doc.py

Each step has one input, one output, and one script. That's the target.

11.4 Debugging Agents — Inversion

When an agent isn't working, flip the question. Instead of "how do I make this work?" ask: **"What would guarantee this fails?"**

Common ways agents fail:

- The directive is ambiguous and the agent interprets it wrong
- A script gets the wrong argument format
- An API key is missing or expired
- The output file path doesn't exist
- The agent skips a step because the directive didn't say it was mandatory

Work through this list before diving into code. You'll find the issue faster.

11.5 Debugging Agents — Ockham's Razor

When something breaks, **the simplest explanation is almost always right.**

Agent not producing output?

→ Check: is the script even running?

→ Check: is the output file being written to the right path?

→ Check: is the API key set in .env?

Before assuming the logic is wrong, rule out the obvious.

Debug in order of simplest-to-most-complex. Don't assume a deep code bug exists until you've confirmed the basics are working.

11.6 Root Cause Analysis

When you fix a bug, don't just fix the symptom — find out why it happened.

Ask "why" three times:

The agent uploaded an empty sheet.

Why? → The script didn't write any rows.

Why? → The input JSON was empty.

Why? → The scrape returned zero results because the API query was malformed.

Root cause: the query format, not the upload logic.

Fix the root cause. Then update the directive to warn about malformed queries. Now the whole system is smarter.

11.7 Helping the Agent Debug

When an agent is stuck or producing wrong output, your job as the human is to give it better signal — not do the work for it.

Effective ways to unblock an agent:

Situation	What to do
Wrong output format	Show it a concrete example of what you expect
Script error it can't fix	Paste the full stack trace, not just "it errored"
Agent is going in circles	Explicitly tell it which step to start from
Directive is ambiguous	Clarify the ambiguous part in plain language
API behaving unexpectedly	Share the raw API response so it can see what's coming back

The more concrete the information you provide, the faster the agent converges on the right answer.

11.8 Systems Thinking — Agents Don't Work in Isolation

Every script you write connects to others. A bug in one step can silently corrupt downstream steps.

Think about your agent as a pipeline, not a collection of independent scripts:

```
scrape → filter → enrich → upload → notify
```

If the filter step drops good leads (silent bug), the enrich step will under-perform, the sheet will look thin, and the notification will say "done" — but the actual deliverable is wrong.

Build in checkpoints. After key steps, have the agent verify the output before continuing. The lead scraping directive does this: run a test scrape of 25 leads, verify 80% match the target industry, then proceed. That checkpoint catches bad data early — before you've scraped 5,000 wrong leads.

11.9 The Map Is Not the Territory

Your directive is a model of how the task should work. Reality will differ.

APIs change. Rate limits are hit. Data comes back in unexpected formats. The directive is a starting point, not a guarantee.

This is exactly why self-annealing matters. Every time reality diverges from the directive, you update the directive. Over time, the map gets more accurate.

Don't get frustrated when an agent follows the directive and still fails — it means the directive needs to be updated with real-world knowledge. That's normal. That's the process.

11.10 The Adjacent Possible — Build Incrementally

Don't try to build your most ambitious agent first. Build the next logical step from where you are.

```
You have: a working web scraper  
Adjacent possible: add email enrichment to the scraped data  
Next step: hook it up to a Google Sheet  
Then: add a Slack notification when it completes  
Then: schedule it to run daily
```

Each step is small and testable. By the time you've taken six adjacent steps, you have a sophisticated automated pipeline — and you understand every part of it because you built it incrementally.

What to Build Next

Start with something small and concrete:

1. Pick one task you do manually and repetitively
2. Write a directive for it (plain English SOP)
3. Find or write a script that handles the core API call
4. Test it with Copilot
5. Iterate — add edge case handling, improve reliability

The framework scales naturally. Once you have 3-4 working directives and scripts, you have a full agent.